

《Agent 专题》课后作业——调试过程与心得 (初稿)

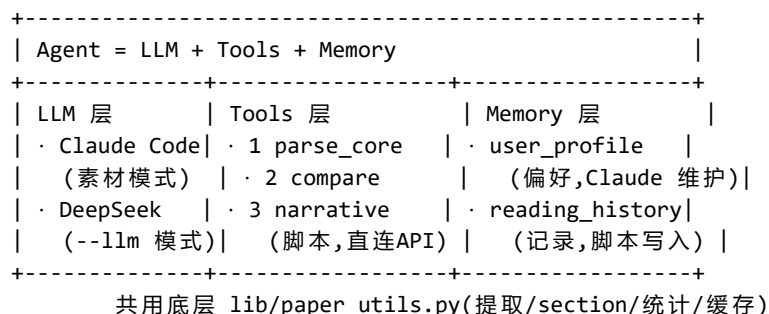
产品: **Paper Reader** 学术论文阅读助手 (Claude Code Skill) 姓名: 杨智 学号: 23009201199 | 命名: 姓名-学号-Agent 作业 源码: livanivter/agent-homework-2026.8.11 说明: 本报告为调试过程初稿 (2026-08-11 更新), 聚焦开发/调试全过程、测试用例与心得; 产品完整设计见同仓库 **实现计划.md**。

一、产品一句话

面向研究生科研场景的学术论文阅读 Agent: 把一篇论文变成结构化卡片 (工具 1)、对比 2~5 篇论文 (工具 2)、剖析论文“为什么这么写” (工具 3), 并具备记忆能力。 **Agent = LLM + Tools + Memory**, 提供两条运行路径:

- 自包含 **ReAct Agent(agent.py)**: 自己写的 Agent 循环, LLM 用 DeepSeek(function calling), 注册 5 个工具 (3 个论文工具 + read_memory/update_profile), 记忆以工具形式参与决策——不依赖任何第三方 Agent 框架。
- **Claude Code Skill**: Claude Code harness 提供 Agent 循环, 三个脚本作为工具, 记忆为 memory/ 下两个 JSON。

二、整体架构 (先交代, 便于理解后续调试)



关键决策: 工具脚本只做“确定性、不容易错的事” (**PDF** 解析、**section** 切分、硬统计), 推理交给 **LLM** 层。这让工具不依赖特定模型、可独立测试。

三、调试过程 (按阶段, 含真实遇到的问题)

阶段 0 • 骨架与共用底层

- 搭建 Skill 目录 (SKILL.md / scripts/ / references/ / memory/), git init。
- 实现 **paper_utils.py**: 文件校验 -> 文本提取 (多级回退)-> section 定位 -> 硬统计 -> 卡片缓存 -> 记忆日志。

阶段 1 • 三个工具初版

- 三个脚本各自实现, 共用 **paper_utils**。
- 初版 section 定位用“整行正则匹配标题” (标题行 <=90 字符、以 Introduction 等结尾)。

阶段 2 • 真实论文测试, 暴露四个问题

问题 1: Windows 下脚本输出编码不是 UTF-8(最隐蔽、最要命)

- 现象: 脚本输出中文在重定向后乱码。
- 排查: 检查输出文件的字节, 发现 0xcc ——这是 GBK 中文字节。Windows 上 Python 的 stdout 被重定向时, 默认用 locale 编码 (**GBK**) 而不是 **UTF-8**, 而下游 (Claude/终端/日志) 按 UTF-8 读, 必乱码。
- 修复: 新增 **ensure_utf8_io()**, 在三个脚本 main 开头强制 **sys.stdout.reconfigure(encoding="utf-8")**。
- 验证: 重定向后文件确认为 UTF-8, 中文字段正常。
- 教训: 跨平台工具的第一步是统一字节编码, 这比业务逻辑更容易踩坑。

问题 2: 两栏 PDF 的 section 定位整篇失败

- 现象: 对 FedMD(arXiv 单栏) 能切出 7 节; 对 FedProto(AAAI 两栏) 提取正常 (67260 字符) 但只返回 `fulltext`, 一节都没切出来。
- 排查: 打印标题行, 发现长这样——
`' Introduction Swersky, and Zemel 2017; Liu et al. 2020; Dvo...' (133字符)`
- 根因:`pdftotext -layout` 会把左右两栏混排在同一物理行, 标题 (“Introduction”) 和另一栏正文挤在一行, 整行远超 90 字符限制, 正则整行匹配失败。
- 方案对比: 测试 `pdftotext` 不带 `-layout`(raw 模式), 发现两栏论文在 `raw` 模式下正文更连贯 (Abstract/Introduction 都是独立整行)。
- 修复 (双管齐下):
 1. `extract_text` 改为先 `raw` 模式、空输出再回退 `-layout`;
 2. `detect_sections` 改为 “整行 ≤ 90 直接判; 超长则按 ≥ 3 个空格切成块, 逐块判标题”, 兼容单栏/两栏。
- 验证:FedMD / FedProto / ResNet / C2KD 四篇全部正确分节。
- 教训: 两栏排版 (会议论文常见) 是真实世界最常见的 **PDF** 形态, 必须专门处理。

问题 3: 参考文献污染” 结论” 节

- 现象:FedMD 结论节篇幅高达 43%、引用 20 条——其实那 20 条引用全是参考文献列表。
- 根因: 没有识别 **References** 标题, 参考文献被并进最后一个检测到的节 (结论)。
- 修复:SECTION_ALIASES 增加 `references / bibliography / works cited` 独立分节。
- 验证: 参考文献独立成节 (35%), 结论恢复干净的 5%,we 词/声称词统计也随之正确。

问题 4: 同领域论文的重叠硬信号为 0.0

- 现象:FedMD vs FedProto 两个联邦学习论文,Jaccard 关键词重叠显示 0.0, 信号失效。
- 排查: 追到 FedProto 的 abstract/intro 取不到内容——因为问题 2 没修好时 `section` 检测失败, 字段全空。
- 修复: 问题 2 修好后, 重叠信号自然恢复: 同领域 **0.212**, 跨领域 (ResNet vs 联邦学习)**0.081**, 有了区分度, 能作为 LLM 判断” 可比性” 的客观依据。
- 教训:bug 可能藏在另一个 bug 后面; 修好上游, 下游信号会自愈。

阶段 3 • LLM 接入与 DeepSeek

- 初版脚本只认环境变量; 应需求改为支持项目根 `.env` 文件 (环境变量优先), 提供 `.env.example` 模板。
- 用户配置 `LLM_PROVIDER=deepseek` 但把 key 误放 `ANTHROPIC_API_KEY`。**DeepSeek** 走 **OpenAI** 兼容协议, 与 Anthropic 协议不通——新增 `deepseek provider`(OpenAI 兼容, `DEEPSEEK_API_KEY`, 默认模型 `deepseek-v4-flash`), 实测官方 API 返回正常。
- 增加 `DEEPSEEK_BASE_URL` 覆盖项, 支持网关/代理场景。
- 验证:`parse/compare/narrative` 三工具 `--llm` 全流程跑通。

阶段 4 • 自包含 ReAct Agent 与记忆闭环 (关键升级)

背景: 用户质疑” 三个会调 LLM 的脚本算不算 agent, 记忆没体现”。确实——单次 `--llm` 调用没有循环、不是 agent;`reading_history` 只是日志、不改变行为。

方案: 新增 `agent.py`, 自己写 ReAct 循环 (核心循环约 15 行, 整脚本 228 行, 呼应课件 Agent Loop 伪代码): - LLM = DeepSeek, 走 OpenAI 兼容 **function calling**; - 注册 5 个工具:`parse_paper / compare_papers / analyze_narrative / read_memory / update_profile`; - 循环:LLM 决策调哪个工具 -> 执行 -> 结果放回上下文 -> 再决策 -> 直到无工具调用。

实测记忆闭环 (两轮对话):

任务1: "对比 FedMD 和 FedProto, 把 3 条关键差异记进记忆"

-> [step1] 并行调用 `read_memory + compare_papers`

-> [step2] 调用 `update_profile`, 把 3 条差异 + 偏好领域写入 `user_profile.json`

任务2: "我上次让你对比了什么? 直接读记忆回答"

-> [step1] 调用 `read_memory` -> 完整读回上次存的 3 条差异

- 验证:agent 决策正确、记忆真实写入并读回、行为被记忆改变 (第 2 次对话直接基于记忆回答)。
- 这回答了” 算不算 agent、记忆在哪”：循环自己写，记忆以工具形式参与决策，闭环可演示。

阶段 5 • 用户实测发现：记忆不存文件路径，每轮都要重输路径 (2026-08-11)

现象 (用户实测)：多轮对话对比不同年份的联邦学习论文 (如 FedGKT 2020 vs FedCMD 2025) 时，每次提问仍要写完整文件路径。记忆里虽然存了对比结论、偏好、甚至论文库路径，但 agent 不会从记忆自动补全路径——记忆无法直接驱动下一次工具调用。

复现：

任务1: "对比 FedGKT 和 FedCMD"

-> agent 需要完整路径,用户手输 ...\raw论文\FedGKT_2020.pdf 与 ...\FedCMD_2025.pdf

任务2: "再对比 FedGKT 和 FedProto"

-> 用户仍需重新手输 FedGKT 的完整路径 (记忆里没有"论文名->路径"的映射)

根因分析: 1. 记忆存的是”事实”，不是”可操作的引用”。user_profile 的 user_notes 虽记了论文库路径，但那是文本说明，不是论文名 -> 绝对路径 的结构化映射,agent 无法拿它做工具参数补全。2. 论文名到文件的映射缺失。用户提问用论文名 (“FedGKT”), 文件名却是 FedGKT_2020.pdf;agent 不知道 FedGKT 对应哪个文件，只能回头问路径。3. 记忆与工具调用参数之间没有打通。read_memory 返回的内容 (含路径) 进了上下文，但 agent 构造 file_path 参数时，没有”从记忆自动补全路径”的逻辑——记忆能”回读”，不能”驱动”。

设计教训：记忆不仅要存”结论/偏好”，更要存”可操作的引用” (文件路径 / ID / 链接)。否则记忆只是”聊天记录”，不是”工具的参数来源”。这是 Agent 记忆设计里”该记住什么”的关键问题，直接暴露了当前记忆 schema 的设计缺口。

后续修复方向 (本次不改代码，仅记录)：- user_profile 增加 paper_paths 字段:{"FedGKT": "C:\\...\\raw 论文\\FedGKT_2020.pdf"},update_profile 时由 agent 或脚本自动登记; - 新增 find_paper 工具: 输入论文名，扫描论文库目录返回匹配的绝对路径，让 agent 免手输; - read_memory 暴露论文库根目录,agent 按 根目录 / 论文名.pdf 拼接。

四、测试用例与结果 (作业第 2 题素材)

类别	用例	结果
√ 成功	单栏论文 (FedMD)7 节全切分	通过
√ 成功	两栏论文 (FedProto)8 节全切分	通过
√ 成功	parse --llm 产出五层方法卡片 (中文, 含 L0 核心思想/L4 范式标签)	通过
√ 成功	compare --llm 判定 FedMD<->FedProto 为”部分可比” (判断正确)	通过
√ 成功	narrative --llm 产出 8 维叙事地图 + 引用证据	通过
√ 成功	agent.py ReAct:step1 并行调用 read_memory+parse_paper -> 产出卡片	通过
√ 成功	agent.py 记忆闭环: 对比->update_profile 写入-> 下次 read_memory 读回	通过
√ 成功	agent.py 工具失败容错: 路径错误时 agent 主动请求绝对路径	通过
! 边界	对比 1 篇 (不足下限)/ 6 篇 (超上限)	明确拒绝
! 边界	文件 >50MB	拒绝
! 边界	--llm 但未配置 key	明确提示, 不崩溃
! 边界	多轮对话: 记忆不含”论文名->路径”映射	每轮需重输完整路径 (设计缺陷, 修复方向已记录)
× 失败	文件路径不存在	报错”文件不存在”
× 失败	非 PDF 文件	报错”仅支持 PDF”

类别	用例	结果
× 失败	加密/纯扫描版 PDF	报错”无法提取文本”，不编造内容

五、提示词清单（作业要求附全部提示词）

5.1 用户实际输入的提示词（完整逐条记录）

整个开发采用 Vibe-Coding。以下为用户实际输入的全部提示词，按出现顺序记录：

1. "C:\Users\Double\Desktop\Agent 专题-季玮晔.pdf" 你看一下这个里面的作业，你觉得实现起来麻烦吗
2. 有哪些容易实现的垂直领域呢
3. 就学术论文阅读助手这个放向，具体工具就写解析论文核心内容，论文内容对比，论文叙事剖析（为什么要这么写），这三个工具，你来给出这三个工具的具体设想，我看一下思路
4. 对比这个功能，我想同时对比多个文论，不一定是两个，有些论文方向区别很大，存在较大偏差，我希望这样的情况也可被比较，工具一的方法描述要尽可能详细，你再完善一下思路计划
5. N 篇对比上限 4-5 篇即可，思路大体没有问题，你这个工具要最后是封装到 skill 里吗
6. 那我还有个问题，这个让自己设计 agent，这个是要自己写 agent 循环吗
7. 那你把这个实现计划写成一个.md 文件，存在"C:\Users\Double\Desktop\work"，防止你后面忘记了，写完计划后，在这个路径下创建项目，开始实现三个工具 + JSON 记忆 + Claude Code Skill
8. 配置 api key 在那个文件里，具体在哪行
9. "C:\Users\Double\Desktop\work\paper-reader-skill\.env.example" 这个文件是干嘛的
10. 帮我把调试过程整理成报告初稿，写 md 文件。所以现在完成的是个什么东西，三个 skill？里面的工具函数调用 deepseek？所以你更改了适配？不走 Claudecode？
11. 现在你已经写好了项目，但是我应该如何使用呢，就直接手动运行三个工具??? 给出使用示例
12. 我还是不理解，现在相当于就三个脚本，脚本里面会调用 llm 完成特定的需求，这算自己的 agent 吗，你这个记忆功能也没体现呀
13. 写个 agent 循环,react 循环，模型调用还是 deep seek
14. 你现在重写一下报告，我刚刚调试了一下，这个 agent 记忆的时候不会记忆文件路径，导致每次问都要写路径，你把这个写进调试报告里，写成 md 文件，先不改代码了
15. 做第三题刁难实验，你自己试一下，你有哪些功能是无法成功实现的，先列个清单，然后我再让你尝试
16. 你现在重新读一下项目，然后看有什么地方和现在的具体实现有什么不一样的地方，修改一下，和目前的实现对齐。
17. 选择连续 7 天，每天自动检查 arXiv 上新发布的联邦学习论文这个方向做刁难，写道报告里，然后转成 pdf

5.2 开发过程中使用的构建/调试提示词

在上一节用户提示词的驱动下，开发过程实际使用的构建/调试提示词如下：

1. 创建 Claude Code Skill 骨架 paper-reader-skill，含 SKILL.md/scripts/lib/references/memory,git init
2. 实现 parse_paper_core.py：用 pdftotext 提取文本,section 定位，输出七字段卡片 JSON；共用逻辑进 lib/paper_utils.py
3. 实现 compare_papers.py:2~5 篇，可比性判定，同领域逐字段矩阵/跨领域范式映射，输出 markdown 报告
4. 实现 analyze_narrative.py：硬统计 + 8 维度叙事地图 + overclaim 检测
5. 脚本成功后把处理记录追加进 memory/reading_history.json
6. 写 SKILL.md：触发条件、工具调用命令、工作流（读记忆-> 选工具-> 跑脚本-> 更新记忆）
7. 用真实论文测试三个工具，记录成功/边界/失败输出
8. 修 Windows UTF-8 输出:stdout 重定向默认 GBK，强制 UTF-8
9. 修复两栏 PDF section 定位失败:raw 模式优先 + 按栏块切分判标题
10. llm.py 支持 .env 配置与 deepseek provider(OpenAI 兼容协议)

六、心得与反思

1. 工具与推理的边界是 **Agent** 设计的核心决策。让脚本只做确定性预处理、把推理交给 LLM 层，工具变得可独立测试、不绑定特定模型；这也是”LLM + Tools”分工的本意。
2. 真实世界的坑远多于教科书。两栏排版、Windows 字节编码、PDF 质量参差（扫描版/加密/损坏）是调试主力，这部分恰恰是报告最有价值的素材。

3. 确定性硬信号能给 **LLM** 的定性判断托底。Jaccard 重叠、篇幅占比、词频这些”不会错”的指标,让”可比性”“过度声称”这类主观判断有了客观依据。
4. “承认不可比/不编造”是 **Agent** 的诚实原则。加密 PDF 如实报错、跨领域论文如实声明不可比——这是 **Agent** 设计里最容易忽略、也最能体现水准的部分。
5. 记忆的读写时机决定了记忆是否”活”。任务前读偏好、任务后写历史,配合”读史”回看,才构成完整的记忆闭环。
6. 让记忆成为 **Agent** 的一个”工具”,是”记忆能力”最直观的演示方式。把 `read_memory / update_profile` 注册进工具表,记忆就参与决策循环,而不是只当”日志文件”——这回答了”记忆功能体现在哪”的核心质疑。
7. “三个脚本”和”**Agent**”的区别在循环,不在数量。一次 `--llm` 调用只是”LLM+ 模板”;加上 **ReAct** 循环(决策-> 调工具-> 观察-> 再决策),才算 **Agent**。这也是为什么自包含 `agent.py` 是必要的。
8. 记忆该存”可操作的引用”,而不只是”事实”。用户实测发现:记忆存了对比结论,却没存”论文名-> 文件路径”的映射,导致每次都要重输路径、记忆无法驱动下一次工具调用。这是 **Agent** 记忆设计的核心教训——路径 / ID / 链接这类引用要一并记住,记忆才”可操作”。

七、第 3 题:刁难成熟 **Agent** 产品——实测清单与结果 (2026-08-11)

7.1 选定的刁难题

主推:「连续 7 天,每天自动检查 **arXiv** 上新发布的联邦学习论文,把值得看的存进记忆,周末给我一份周报。」

- 失败点: 跨天自主持续运行。当前 **Agent** 没有独立守护进程,会话结束即停止;定时任务仅会话内触发、7 天自动过期;无人值守时无法”醒来”执行。
- 本质: 这是 **Agent** 在时间维度的短板 (正属 **Agentic Infra** 的核心议题: 持久执行 / 调度),而非单次任务能力不足。对 **Claude Code** 同样成立——任何”关闭终端后继续工作”的需求都会失败。

备选 (对本产品最贴题):「查这篇论文在 **Google Scholar** 的引用数」——**Google Scholar** 被墙访问失败;虽可用 **Semantic Scholar** 等替代,但拿不到 **Google Scholar** 口径的数字。